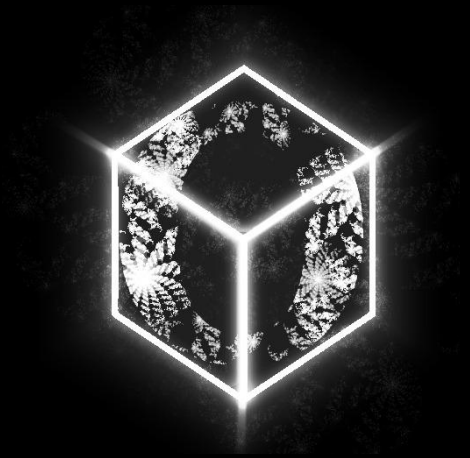
GODOT: VERSIONAMENTO DE CÓDIGO

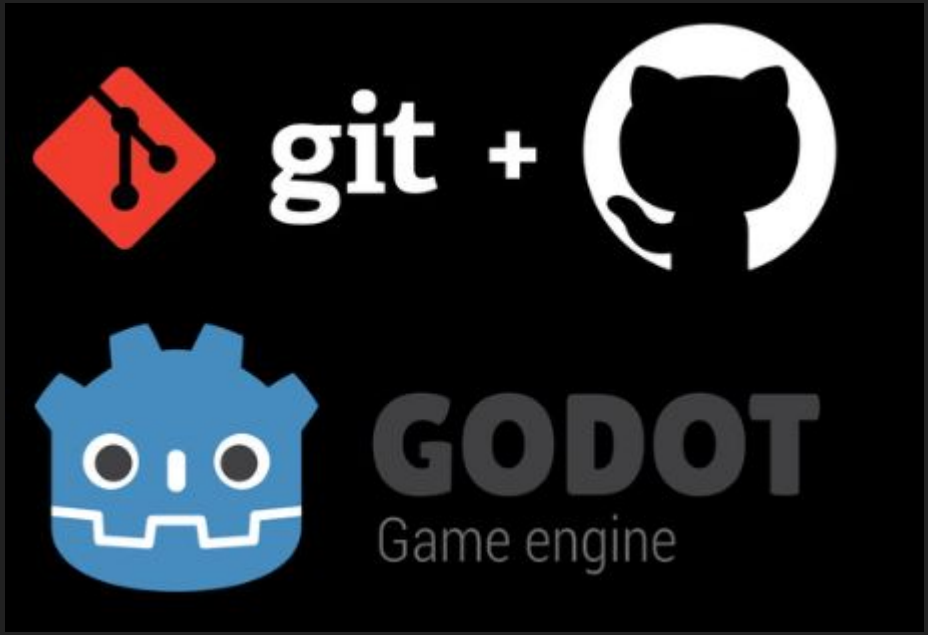
Grupo de Jogos Obscura

Desenvolvido por: Yuji



Introdução

Essa apresentação tem como objetivo apresentar, de forma clara e organizada, os conceitos de versionamento de código utilizando o **Git** e **Github**, com focos em projetos desenvolvidos na **Godot Engine**.



Referência

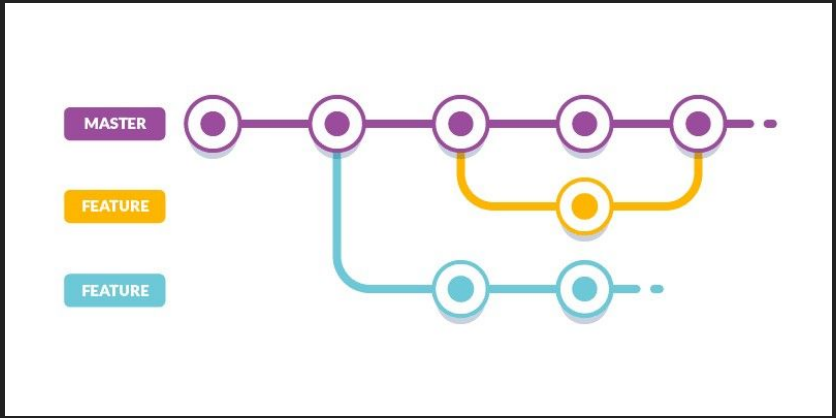
A referência principal dessa pesquisa é do vídeo do canal **Guará Programador**:

https://youtu.be/1vWzOnu4Nq0?si=Sbcl9X3bTgBc1agU

O que é versionamento de código?

O versionamento de código é crucial para desenvolvedores de código e equipes trabalharem de forma eficiente. Ele permite:

- Rastrear todas as alterações feitas no código
- Recuperar versões anteriores em caso de erro
- Trabalhar em equipes simultaneamente no mesmo projeto



Por usar o Git?

O **Git** é um sistema de **controle de versão** distribuído que permite **rastrear todas as alterações feitas** em seu código ao longo do desenvolvimento.

Por **exemplo**, se um estagiário cometer um erro na versão atual, o trabalho não estará perdido, pois é possível retornar para as versões anteriores de seu projeto.

Principais vantagens: controle do histórico, possibilidade de desfazer alterações, trabalho colaborativo em equipe e versionamento local sem depender da internet.

Por que usar o Github?

Para compartilhar o nosso projeto Git, precisamos de uma plataforma de hospedagem como o **Github**. Ele é um serviço onde que guarda os repositórios de códigos criados em Git. Ele funciona como:

- Armazenamento em nuvem para projetos
- Ferramenta de colaboração em equipe
- Histórico visual de commits e alterações

É conhecido como uma "***rede social de devs***".

Como fazer o versionamento de código do Godot?

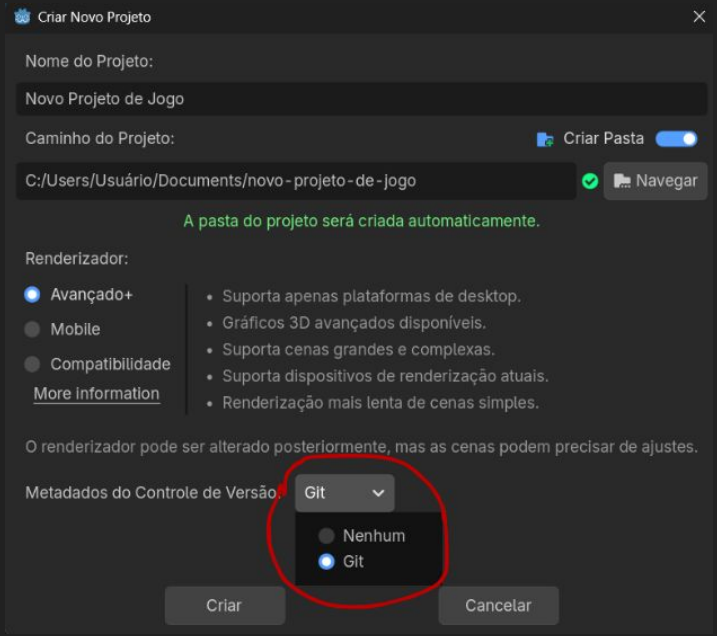
Primeiro, vamos instalar o **Github Desktop**. É uma ferramenta que **gerencia o Git**, **integrada diretamente com o Github**, sendo uma alternativa para subir o código no repositório de maneira simples.

https://desktop.github.com/download/

Como colocar o Git no projeto Godot?

Ao criar um novo projeto no Godot, selecione:

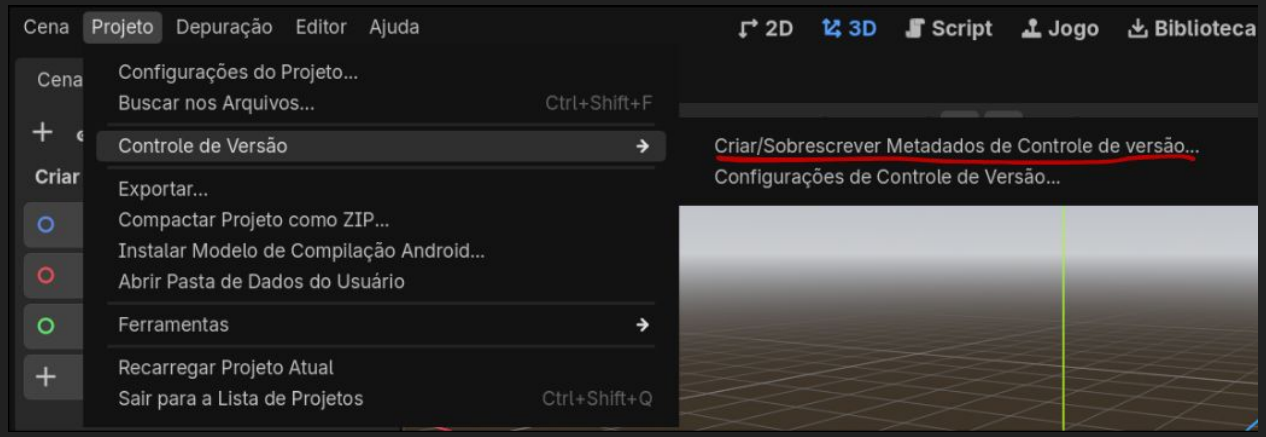
Controle de Versão → Git



Como colocar o Git no projeto Godot?

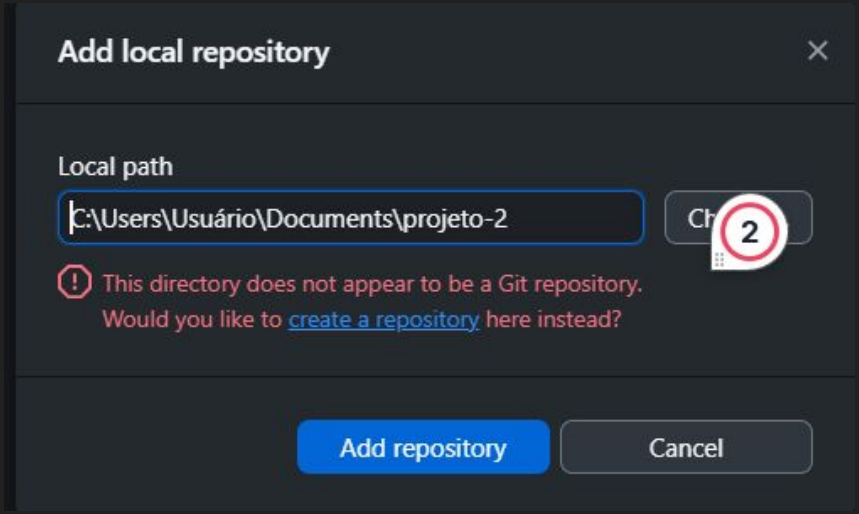
Se você já tem um projeto existente, basta clicar em:

Projeto → Controle de versão → Criar/Sobrescrever Metadados de Controle de Versão



Criando repositório local

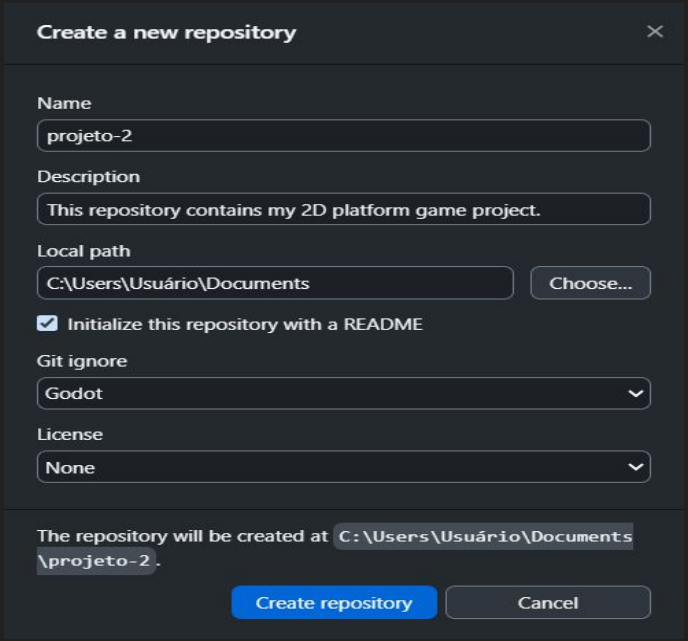
1. Abra o **Github Desktop**
2. Clique em: **File** → **Add local repository**
3. Selecione a pasta do projeto
4. Clique em **create repository**



Criando repositório local

Ao criar:

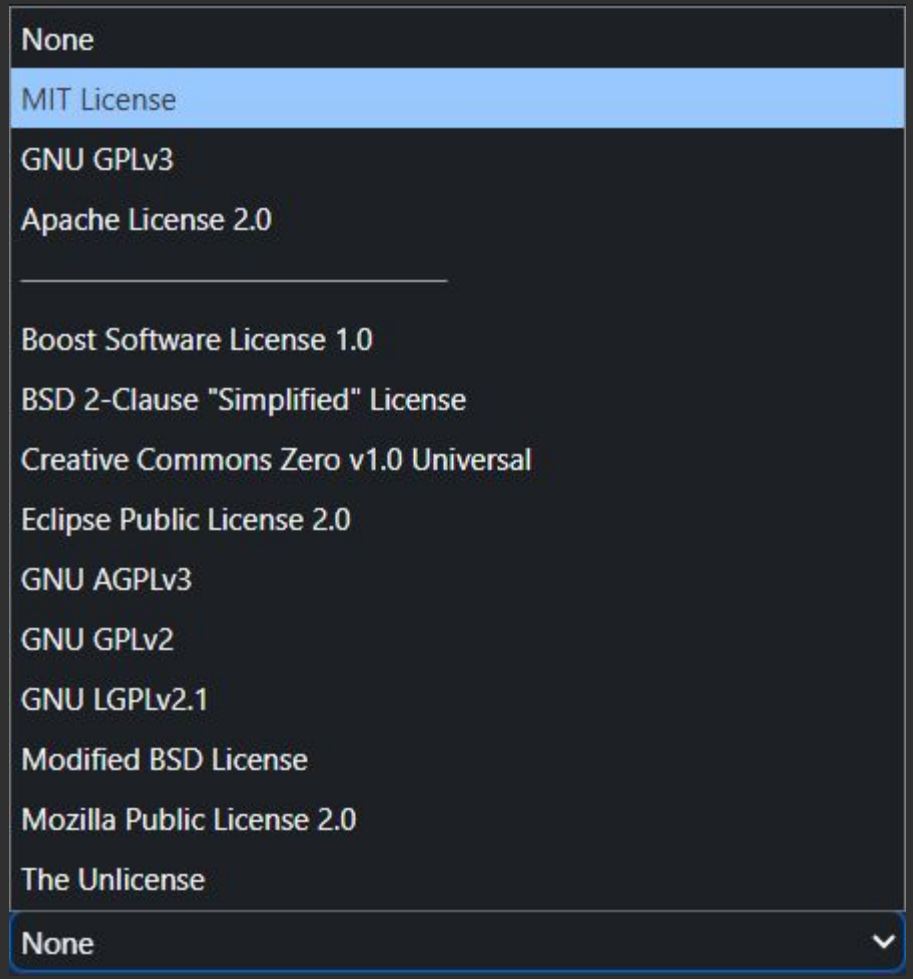
- Defina o nome e descrição
- Crie um README se necessário
- Adicione um .gitignore (selecione Godot)
- Escolha uma licença



Licença

A **licença** é um essencial para o seu repositório, pois todo código é considerado uma propriedade intelectual e precisamos de um documento que reconheça o autor e regule o seu uso.

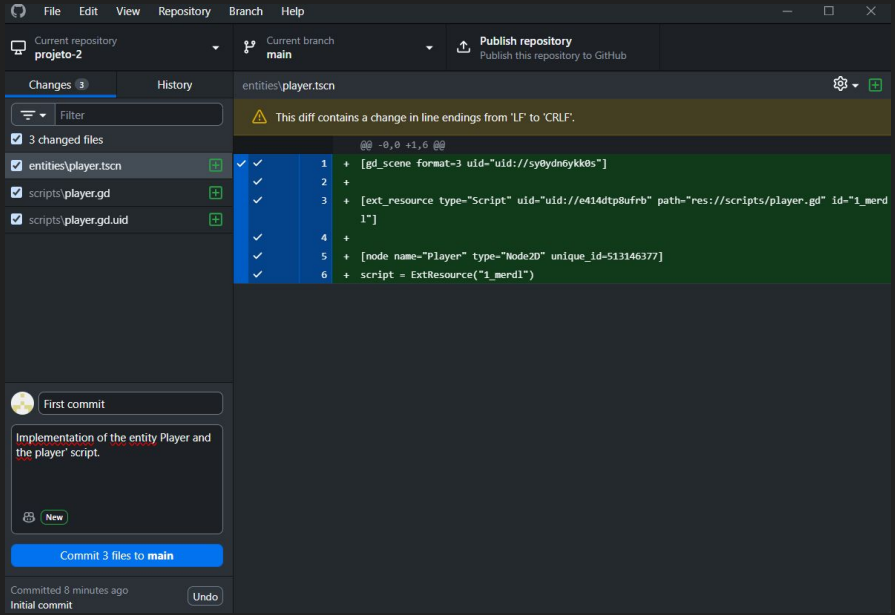
A recomendação é usar a **MIT License**, por ser simples e permissiva.



Publicando repositório no Github

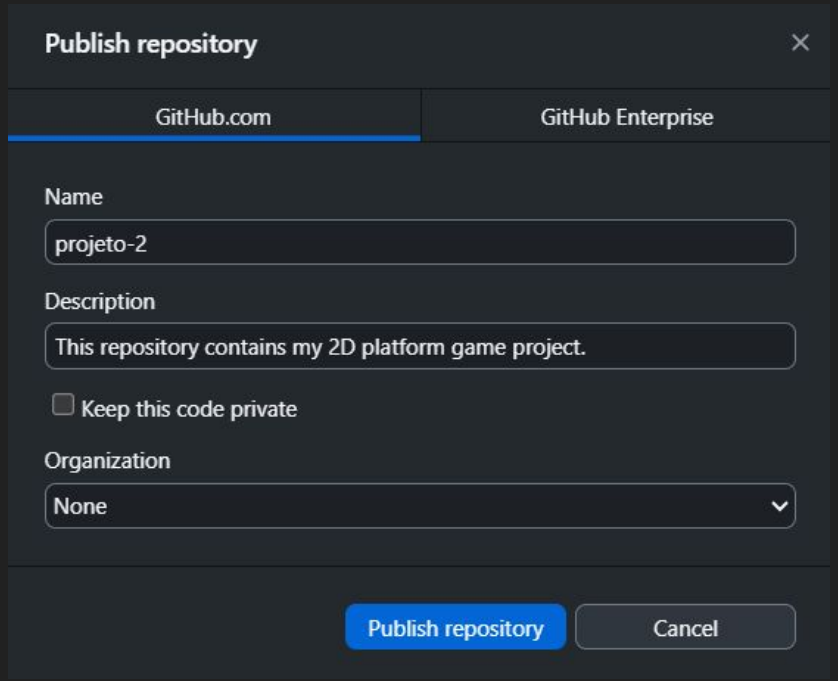
Agora com o repositório criado, podemos controlar os arquivos do seu código e publicá-lo no Github:

1. Vá até a aba **Changes**
2. Registre as alterações com um **commit**
3. Clique em **Publish Repository**



Publicando repositório no Github

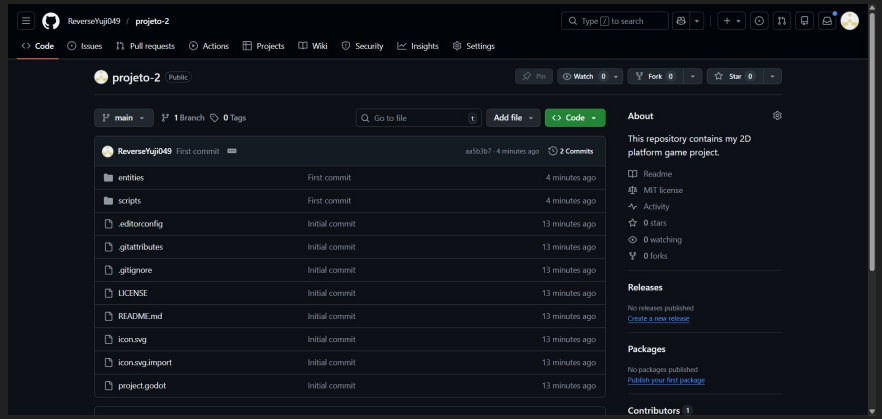
Ao clicar em Publish repository, preenche o nome do repositório e sua descrição. Também é possível colocá-lo como privado e atribuído a alguma organização.



Atualizações do Repositório

Para registrar as atualizações, deve-se criar um **Commit** com as mudanças e arquivos selecionados.

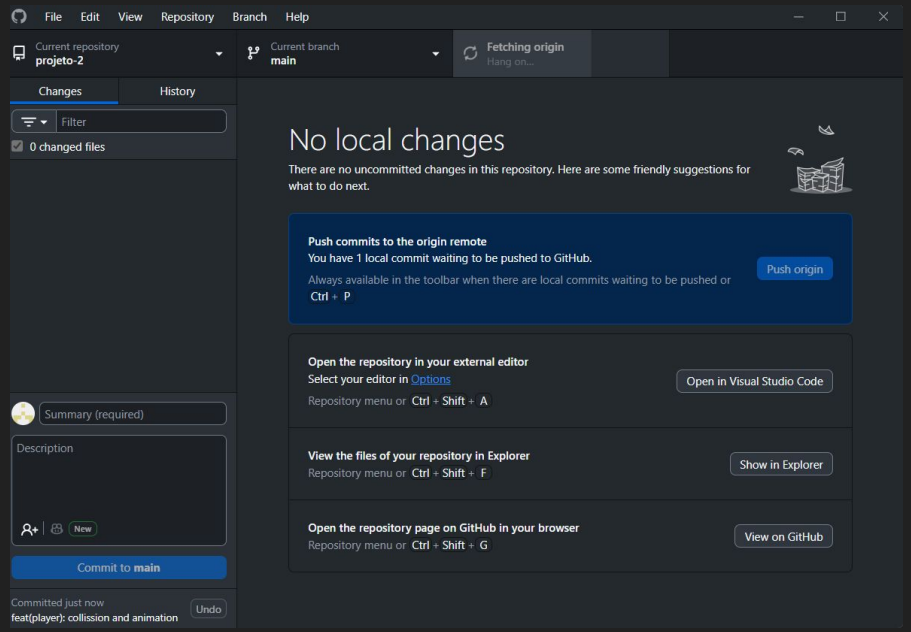
Ao registrá-lo, ele aparecerá no **History** (Histórico). Precisamos atualizar o repositório Github com essas mudanças locais.



Atualizações do Repositório

Quando houver qualquer alteração do Github que não seja na sua máquina local, é preciso pedir permissão no Github Desktop, clicando em **Fetch origin**.

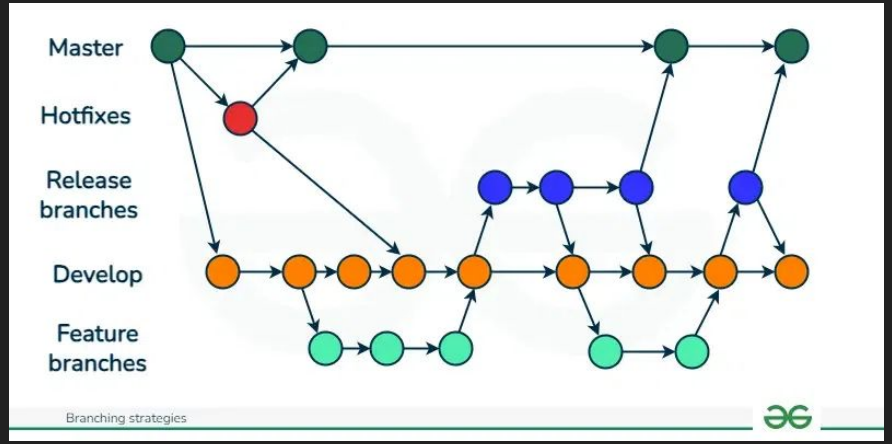
Se houver atualizações na nuvem, aparecerá o **Pull origin** para puxá-las ao repositório local.



Integração no Trabalho de Equipe

Vamos supor que você precisa dividir as tarefas entre integrantes de sua equipe e que cada um esteja encarregado de uma função específica.

Branches

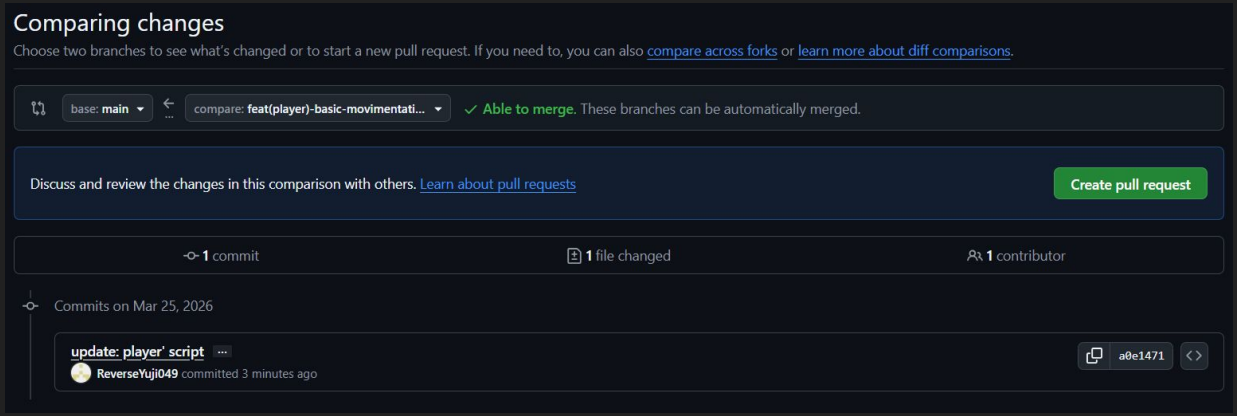


Branches são bifurcações/ramificações de uma árvore que contém diferentes versões do seu código. Todo projeto Git começa com a **branch main** que contém versões estáveis do produto. Como estamos produzindo recursos para o jogo, é recomendável criar uma **branch separada** para unir futuramente à main.

Merge e Pull Request

No **Github Desktop**, podemos ver qual **branch** você está e criar uma nova branch. Assim qualquer alteração no código será registrada nesta branch selecionada.

Para integrar essas mudanças da nova branch na principal, basta criar um **Pull Request** no **Github**:



Merge Conflict

O **Github** irá verificar se o **merge** (mesclagem) é possível. Se não for possível, pode ocorrer o erro **Merge Conflict** (Conflito de Mesclagem), tendo alterações no mesmo trecho e na mesma seção do código em diferentes commits. Assim, a ferramenta não saberá qual modificação será mantida.

O **Github** permite resolver esses conflitos no botão “**Resolve conflicts**”, porém é recomendado solucionar na própria **máquina local** através de **commits**.

Git Large File Storage

Conforme o crescimento do seu projeto, você provavelmente irá lidar com **arquivos grandes** como **texturas, objetos 3D ou arquivos de áudio.** Isso é um problema por conta do limite dos repositórios no Github.

Para resolver isso, temos o **Git Large File Storage** que os-gerencia, criando referências para eles, enquanto os arquivos reais são armazenados separadamente.

https://git-lfs.com/

Git Large File Storage

Infelizmente, não é possível usá-lo no Github Desktop, é necessário usar o terminal.

Repository → Open in Command Prompt

Ao abrir o CMD, utilize os comandos:

- **Instalar o Git LFS:** git lfs install
- **Rastrear determinado arquivo grande:** git lfs track “[NOME]”
  - **Imagens:** git lfs track “*.png”
  - **Áudios:** git lfs track “*.wav”

Conclusão

O uso de **Git** e **GitHub** é fundamental para garantir:

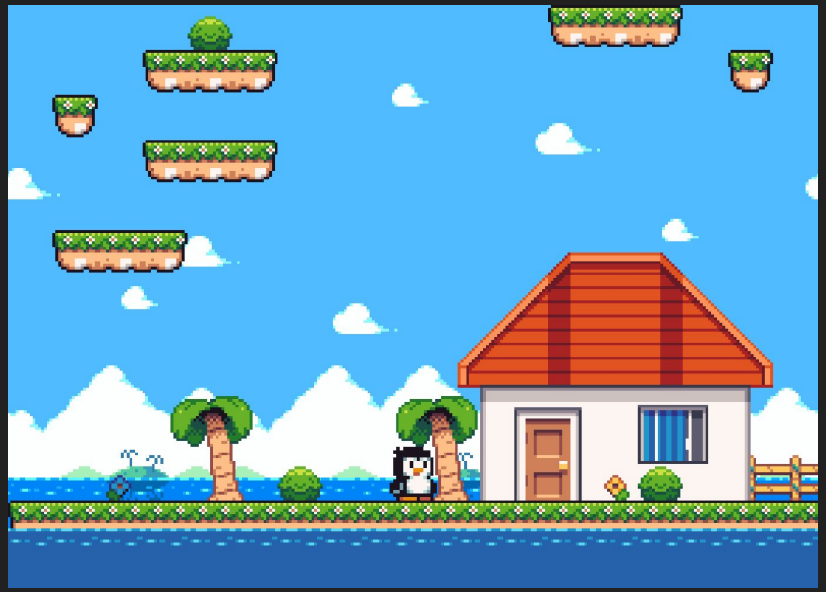
- Organização do projeto
- Segurança do código
- Colaboração eficiente em equipe

A utilização dessas ferramentas permite que o desenvolvimento seja mais profissional, escalável e confiável.

Veja também:

Meu repositório contendo estudos de Godot através do curso no Youtube do professor Rafael Forbeck:

https://github.com/ReverseYuji049/projeto-1.git

Meu portfólio Unifil:

https://sites.google.com/edu.unifil.br/pensamentocompyuji/p%C3%A1gina-inicial